

Class Node:

Key

Value

Next

Prev

Constructor(key, value):

Set this.key = key

Set this.value = value

Class LRUCache:

Head

Tail

Map (HashMap)

Capacity

Constructor(capacity):

Set this.capacity = capacity

Initialize map as an empty HashMap

Function get(key):

If key exists in map:

Node = map[key]

removeNode(node)

putOnTop(node)

Return node.value

Else:

Return -1 // Key not found

Function put(key, value):

If key exists in map:

Node = map[key]

Node.value = value // Update value

removeNode(node)

putOnTop(node)

Else:

If size of map $\geq$ capacity:

Remove map entry for tail.key

removeNode(tail) // Remove Least Recently Used node

Create new node with key and value

Add node to map

putOnTop(node)

Function removeNode(node):

prevNode = node.prev

nextNode = node.next

If prevNode is not null:

prevNode.next = nextNode

Else:

Head = nextNode // Update head if node was at front

If nextNode is not null:

nextNode.prev = prevNode

Else:

Tail = prevNode // Update tail if node was at end

Function putOnTop(node):

Node.next = head

Node.prev = null

If head is not null:

Head.prev = node

Head = node

If tail is null:

Tail = node // If list was empty, update tail

Main Function:

Cache = LRUCache(3)

Cache.put(1, 3)

Cache.put(4, 2)

Print cache.get(1) // Output: 3

Cache.put(5, 6)

Print cache.get(7) // Output: -1

Print cache.get(5) // Output: 6

Cache.put(7, 4)

Print cache.get(4) // Output: -1 (Evicted due to capacity)

Print `cache.get(1)` // Output: 3

Print `cache.get(5)` // Output: 6